Lab 4: Functional Dependencies — RLMS

Data Management (M231) · Fall 2026 · UM6P – College of Computing · Professor: Karima Echihabi · Due: Sunday 11/10 at 23:59

Section I — People

Covers Person, its Student and Employee subtypes, the ISA hierarchy below Employee, and the Advises relationship (PE1–PE6).

I.1 Person

Relational schema: Person(pid, firstName, lastName, email, affiliation, startDate, endDate)

ID	Functional dependency	Justification
FD1	`pid → {pid, firstName, lastName, email, affiliation, startDate, endDate}`	Key FD. pid is the primary key; each person has one name, email, affiliation, start date and optional end date (PE1).
FD2 (A1)	`email → {pid, firstName, lastName, affiliation, startDate, endDate}`	Assumption A1: each person has a unique institutional email, so email is a candidate key. By transitivity with FD1 it determines every other attribute. Lab 2 declares email NOT NULL but not UNIQUE, so this holds only under A1.

Candidate key(s): {pid}, {email} (A1)

Considered and rejected (not FDs):

- `{firstName, lastName} → pid`: two different people can share a name.

I.2 Student

Relational schema: Student(pid, program)

ID	Functional dependency	Justification
FD3	`pid → program`	A student is enrolled in exactly one program {BSc, BEng, MSc, PhD} (PE3).

Candidate key(s): {pid}

Note: Student.pid is also a foreign key to Person(pid) (ISA subtype).

I.3 Employee

Relational schema: Employee(pid, phone, office, supervisorPid)

ID	Functional dependency	Justification
FD4	`pid → {pid, phone, office}`	Each employee has one institutional phone number and one office (PE4).
FD5	`pid → supervisorPid`	ReportsTo (PE5): an employee reports to at most one supervisor, and only the current one is kept. NULL for the Dean.

Candidate key(s): {pid}

Considered and rejected (not FDs):

- `office → phone`: employees can share an office but have different phone numbers.
- `supervisorPid → office` or `supervisorPid → phone`: a supervisor's subordinates sit in different offices.
- `phone → pid`: not adopted, because a phone line can be shared (reception, lab extension) and neither the spec nor Lab 2 says it is unique.

I.4 Academic and NonAcademic

Relational schema: Academic(pid) | NonAcademic(pid)

ID	Functional dependency	Justification
FD6	`pid → pid` (trivial)	These relations only record subtype membership (PE4); they have no other attributes.

Candidate key(s): {pid}

I.5 Faculty, Administrative and Technical

Relational schema: Faculty(pid, position) | Administrative(pid, position) | Technical(pid, position)

ID	Functional dependency	Justification
FD7	`pid → position` (in each of the three relations)	A staff member holds one position at a time (PE4). The allowed values differ per subtype, which is why position is stored in the subtype relations and not in Employee.

Candidate key(s): {pid}

I.6 Advises

Relational schema: Advises(academicPid, studentPid)

ID	Functional dependency	Justification
FD8	`{academicPid, studentPid} → {academicPid, studentPid}` (trivial)	Many-to-many link table with no extra attributes (PE6), so the whole row is the key.

Candidate key(s): {academicPid, studentPid}

Considered and rejected (not FDs):

- `studentPid → academicPid`: a student has at least one advisor and may have several (PE6).
- `academicPid → studentPid`: an academic can advise many students.

Note: The rule "every student has at least one advisor" is a participation constraint, not an FD.

Section II — Laboratories, Projects and Budgets

Covers laboratories, project teams, and the budgets that fund them (LA1–LA4, PR1–PR4, BD1–BD3).

II.1 Laboratory

Relational schema: Laboratory(labId, name, building, roomNumber, discipline, supervisorPid)

ID	Functional dependency	Justification
FD9	`labId → {labId, name, building, roomNumber, discipline}`	Key FD. labId identifies one laboratory with one set of descriptive attributes (LA1).
FD10	`labId → supervisorPid`	Supervises (LA4) is many-to-one: each laboratory has exactly one Faculty supervisor, while a Faculty member may supervise several laboratories. The reverse does not hold.
FD11 (A2)	`{building, roomNumber} → labId`	Assumption A2: a given room hosts at most one laboratory, so (building, roomNumber) is a candidate key.

Candidate key(s): {labId}, {building, roomNumber} (A2)

Considered and rejected (not FDs):

- `discipline → building`: a discipline can be spread across several buildings.

II.2 Attached

Relational schema: Attached(pid, labId)

ID	Functional dependency	Justification
FD12	`{pid, labId} → {pid, labId}` (trivial)	Many-to-many link between persons and laboratories (LA3); no extra attributes.

Candidate key(s): {pid, labId}

II.3 ResearchProject

Relational schema: ResearchProject(code, title, startDate, endDate, status)

ID	Functional dependency	Justification
FD13	`code → {code, title, startDate, endDate, status}`	The project code identifies one project with one title, one period and one current status {Proposed, Active, Suspended, Completed, Cancelled} (PR1).

Candidate key(s): {code}

Considered and rejected (not FDs):

- `title → code`: titles are not guaranteed to be unique.

II.4 Participates

Relational schema: Participates(pid, projectCode, role)

ID	Functional dependency	Justification
FD14	`{pid, projectCode} → role`	The role is an attribute of the (person, project) pair, so a person has exactly one role {PI, Co-PI, Collaborator} in a project (PR3, PR4).

Candidate key(s): {pid, projectCode}

Considered and rejected (not FDs):

- `{projectCode, role} → pid`: only PI and Co-PI are limited to one person per project; Collaborator can be held by many. "Exactly one PI, at most one Co-PI" depends on the value of role, so it is a constraint (trigger or application logic), not an FD.

II.5 Budget

Relational schema: Budget(budgetLine, amountGranted, amountDisbursed, startDate, endDate, managerPid)

ID	Functional dependency	Justification
FD15	`budgetLine → {budgetLine, amountGranted, amountDisbursed, startDate, endDate}`	Each budget line has one granted amount, one disbursed amount and one validity period (BD1).
FD16	`budgetLine → managerPid`	Manages (BD2) is many-to-one: each budget is managed by exactly one Academic, while an Academic can manage several budgets.

Candidate key(s): {budgetLine}

II.6 FundsPrj and FundsLab

Relational schema: FundsPrj(budgetLine, projectCode) | FundsLab(budgetLine, labId)

ID	Functional dependency	Justification
FD17	`{budgetLine, projectCode} → {budgetLine, projectCode}` and `{budgetLine, labId} → {budgetLine, labId}` (trivial)	Many-to-many funding links with no extra attributes (BD3), so their only FDs are trivial.

Candidate key(s): {budgetLine, projectCode} and {budgetLine, labId}

Section III — Equipment and Certifications

Covers equipment models and units, certifications, and which people hold them (EQ1–EQ7, CE1–CE7).

III.1 EquipmentModel

Relational schema: EquipmentModel(modelId, commercialName, manufacturer, category, requiredEnvironment, trainingMandatory)

ID	Functional dependency	Justification
FD18	`modelId → {modelId, commercialName, manufacturer, category, requiredEnvironment, trainingMandatory}`	A model is a catalogue entry with one set of characteristics (EQ2).
FD19 (A3)	`{commercialName, manufacturer} → modelId`	Assumption A3: a manufacturer does not release two models with the same commercial name, so this pair is a candidate key.

Candidate key(s): {modelId}, {commercialName, manufacturer} (A3)

Note: By augmentation, `{commercialName, manufacturer, category} → modelId` also holds, but category is redundant there. If the same manufacturer could reuse a name across categories, replace A3 by the three-attribute version.

III.2 EquipmentUnit

Relational schema: EquipmentUnit(serialNo, acquisitionDate, purchaseCost, status, portable, modelId, labId)

ID	Functional dependency	Justification
FD20	`serialNo → {serialNo, acquisitionDate, purchaseCost, status, portable}`	These describe one physical unit identified by its serial number (EQ3, EQ6).
FD21	`serialNo → modelId`	InstanceOf: each unit belongs to exactly one model (EQ5).
FD22	`serialNo → labId`	LocatedIn: each unit is currently in exactly one laboratory, and only the current location is kept (EQ3, EQ4).

Candidate key(s): {serialNo}

Considered and rejected (not FDs):

- `modelId → purchaseCost`, `modelId → portable`, `modelId → status`: units of one model can have different prices, portability and status.
- `modelId → labId`: units of one model can be in several laboratories.

III.3 Certification

Relational schema: Certification(code, title, issuingAuthority, validityPeriod, safetyLevel)

ID	Functional dependency	Justification
FD23	`code → {code, title, issuingAuthority, validityPeriod, safetyLevel}`	A certification type has one title, issuer, validity duration and safety level (CE2).

Candidate key(s): {code}

III.4 Requires

Relational schema: Requires(modelId, certCode)

ID	Functional dependency	Justification
FD24	`{modelId, certCode} → {modelId, certCode}` (trivial)	Many-to-many: a model may require several certifications and a certification may cover several models (CE6).

Candidate key(s): {modelId, certCode}

III.5 Holds (derived attribute)

Relational schema: Holds(pid, certCode, issueDate, expirationDate, grade)

ID	Functional dependency	Justification
FD25	`{pid, certCode} → {pid, certCode, issueDate, expirationDate, grade}`	Only the latest award of each certification is kept per person, so one row per pair (CE4, CE5).

Candidate key(s): {pid, certCode}

Note: FD21 is the only FD in the schema whose determinant is not a key: {certCode, issueDate} determines expirationDate, which makes the stored expirationDate derivable from the certification's validity period.

Section IV — Reservations

Covers the reservation request and the equipment units it books (RE1–RE4).

IV.1 Reservation

Relational schema: Reservation(resId, submissionTS, plannedStart, plannedEnd, purpose, status, madeByPid, projectCode, approverPid)

ID	Functional dependency	Justification
FD26	`resId → {resId, submissionTS, plannedStart, plannedEnd, purpose, status}`	One request with its own submission time, planned window, purpose and status {Pending, Approved, Rejected, Cancelled, Completed} (RE2). submissionTS is when the request was made, not when the equipment is used.
FD27	`resId → madeByPid`	MadeBy: each reservation is made by exactly one person (RE1).
FD28	`resId → projectCode`	For: each reservation is for exactly one research project (RE1).
FD29	`resId → approverPid`	Approves: at most one approver, designated by the dean; NULL if no approval is needed or given yet (RE3).

Candidate key(s): {resId}

Considered and rejected (not FDs):

- `submissionTS → resId`: two requests can be submitted at the same moment.
- `madeByPid → projectCode`: a person can reserve for any project they take part in.
- `projectCode → approverPid`: different reservations of one project can have different approvers.

Note: madeByPid and approverPid both reference Person(pid); projectCode references ResearchProject(code).

IV.2 Reserves

Relational schema: Reserves(resId, serialNo)

ID	Functional dependency	Justification
FD30	`{resId, serialNo} → {resId, serialNo}` (trivial)	A reservation books at least one unit and a unit can appear in many reservations (RE1); no extra attributes.

Candidate key(s): {resId, serialNo}

Section V — Maintenance and Calibration

Covers the two weak entities owned by EquipmentUnit (MA1–MA3, CA1–CA4).

V.1 Maintenance

Relational schema: Maintenance(serialNo, startTS, endTS, type, description, cost, outcome, technicalPid)

ID	Functional dependency	Justification
FD31	`{serialNo, startTS} → {serialNo, startTS, endTS, type, description, cost, outcome, technicalPid}`	Weak entity identified by the owner key plus the partial key startTS (MA2). Once the unit and start time are fixed, the operation, its cost, outcome and the single technician who did it (MA3) are fixed.

Candidate key(s): {serialNo, startTS}

Considered and rejected (not FDs):

- `serialNo → technicalPid`: a unit can be serviced by different technicians over time.

Note: technicalPid references Technical(pid).

V.2 CalibrationRecord

Relational schema: CalibrationRecord(serialNo, calibDate, calibrationType, result, nextDueDate, remarks)

ID	Functional dependency	Justification
FD32	`{serialNo, calibDate} → {serialNo, calibDate, calibrationType, result, nextDueDate, remarks}`	Weak entity identified by the owner key plus the partial key calibDate (CA3, CA4). One calibration per unit per date, with one result and one next due date (CA2). No technician is recorded (CA1).

Candidate key(s): {serialNo, calibDate}

Considered and rejected (not FDs):

- `serialNo → nextDueDate`: a unit has a history of calibrations with different due dates.

Section VI — Consumables, Suppliers and Stock

Covers consumables, suppliers, supply prices, per-laboratory stock and consumption (CO1–CO10).

VI.1 Consumable

Relational schema: Consumable(consId, name, unitOfMeasure, hazardLevel, reorderThreshold)

ID	Functional dependency	Justification
FD33	`consId → {consId, name, unitOfMeasure, hazardLevel, reorderThreshold}`	One value of each attribute per consumable; the reorder threshold applies in every laboratory (CO2).

Candidate key(s): {consId}

VI.2 Supplier

Relational schema: Supplier(suppId, name, contactEmail, phone)

ID	Functional dependency	Justification
FD34	`suppId → {suppId, name, contactEmail, phone}`	Each supplier has one identifier, name, contact email and phone number (CO3).

Candidate key(s): {suppId}

Considered and rejected (not FDs):

- `name → suppId`: two suppliers may share a trade name.
- `contactEmail → suppId` and `phone → suppId`: not adopted. Neither the spec nor Lab 2 says they are unique, and generic or shared addresses and switchboards are common. They would become valid FDs only if the team adds a matching assumption and a UNIQUE constraint.

VI.3 Supplies

Relational schema: Supplies(suppId, consId, labId, unitPrice)

ID	Functional dependency	Justification
FD35	`{suppId, consId, labId} → unitPrice`	The price belongs to the whole triple: a supplier may charge different laboratories different prices for the same consumable (CO4).

Candidate key(s): {suppId, consId, labId}

Considered and rejected (not FDs):

- `{suppId, consId} → unitPrice`: this is why the relationship is ternary.

VI.4 Stocks

Relational schema: Stocks(labId, consId, quantityOnHand, lastRestockDate, storageCondition, technicalPid, monitoringSince)

ID	Functional dependency	Justification
FD36	`{labId, consId} → {labId, consId, quantityOnHand, lastRestockDate, storageCondition}`	The stock is the quantity of one consumable in one laboratory (CO6, CO7).

ID	Functional dependency	Justification
FD37	`{labId, consId} → {technicalPid, monitoringSince}`	Monitors: each laboratory-consumable stock is monitored by exactly one technician since a given date (CO8).

Candidate key(s): {labId, consId}

Considered and rejected (not FDs):

- `technicalPid → monitoringSince`, `labId → technicalPid`, `consId → technicalPid`: a technician monitors several stocks, starting on different dates.

VI.5 Consumes

Relational schema: Consumes(resId, labId, consId, quantityUsed)

ID	Functional dependency	Justification
FD38	`{resId, labId, consId} → quantityUsed`	The quantity consumed is recorded per reservation and per laboratory stock (CO9, CO10).

Candidate key(s): {resId, labId, consId}

Considered and rejected (not FDs):

- `{resId, consId} → quantityUsed`: the same consumable can be drawn from several laboratories for one reservation, which is why labId is in the key.

Note: (labId, consId) is a foreign key to Stocks(labId, consId).